

RUGSCREENER: Leveraging Temporal Graph Neural Network for Rugpull Detection in DeFi

Cong Wu^{1b}, Hangcheng Cao^{1b}, Jing Chen^{1b}, *Senior Member, IEEE*, Xiyu Yan, Guowen Xu^{1b}, Ziming Zhao, Yang Liu^{1b}, *Senior Member, IEEE*, and Hongbo Jiang^{1b}, *Senior Member, IEEE*

Abstract—The advent of decentralized finance has ushered in a transformative era in the financial sector, leveraging blockchain technology to facilitate peer-to-peer transactions without traditional intermediaries. Amidst this innovation, the DeFi landscape faces the pervasive threat of rugpulls, where developers abruptly abandon projects post-fundraising, leaving investors with devalued assets. This growing concern highlights a critical research gap in the proactive detection and prevention of such fraudulent schemes. To combat this, we propose **RUGSCREENER**, a temporal graph neural network-based solution to identify rugpull risks within DeFi transactions. It employs a dynamic representation of blockchain interactions, enriched with comprehensive node attributes and effective temporal graph learning techniques based on memory and attention mechanisms, effectively capturing the rapid-moving and complex transaction patterns indicative of potential fraud. Our evaluation is based on a newly compiled Ethereum dataset that includes two subsets: an unlabeled set with 1,882,114 transactions from 29,595 tokens for temporal graph representation learning, and a labeled set with 128,819

transactions from 1,000 tokens (500 rugpull and 500 benign) for downstream evaluation. Using this dataset, **RUGSCREENER** achieves a balanced accuracy of 95.7% in detecting rugpull tokens. Remarkably, **RUGSCREENER** surpasses existing state-of-the-art graph learning models in detecting rugpull tokens with enhanced accuracy and reliability.

Index Terms—Rugpull detection, graph neural network, ERC20 tokens, decentralized finance.

I. INTRODUCTION

DECENTRALIZED finance (DeFi), fueled by blockchain technology and smart contracts [1], [2], [3], [4], has significantly transformed the traditional financial landscape by facilitating peer-to-peer financial services without the need for centralized intermediaries [5], [6], [7], [8], [9], [10]. Despite its innovative contributions, the rapid expansion of DeFi has also unveiled substantial security concerns, particularly the emergence of *rugpull* tokens [11], [12]. These schemes involve developers abruptly abandoning a project after fundraising, leaving investors with tokens that are significantly devalued or entirely worthless. In 2023, cryptocurrency users lost nearly \$2 billion to scams, rugpulls, and hacks, with rugpulls alone accounting for approximately \$136 million across 263 incidents [13]. This growing trend underscores the critical need for effective strategies to not only detect but also predict future rugpull risks, thereby protecting the integrity of the DeFi ecosystem and preventing financial losses [11], [14], [15], [16], [17], [18], [19], [20].

A. Existing Efforts

In the rapidly evolving DeFi landscape, the detection of rugpull tokens remains a significant challenge. Existing methods, predominantly traditional machine learning-based [14], [17], [18] and rule-based [19], [20] approaches, exhibit limitations in effectively identifying these fraudulent tokens. Transaction-based strategies, while adept at analyzing patterns in token transfers, often rely on historical data, which may not capture the evolving nature of rugpull schemes [21]. Contract-based methods, focusing on smart contract code vulnerabilities, are effective for certain scams like Ponzi schemes but fall short in detecting rugpulls, which typically exhibit benign contract behaviors [7], [22], [23].

The pseudonymity of blockchain and the dynamic nature of DeFi transactions present significant challenges for fraud detection [24]. Traditional learning-based models often struggle to adapt to evolving attack patterns, while rule-based

Received 2 February 2025; revised 11 May 2025 and 30 July 2025; accepted 2 October 2025. Date of publication 14 October 2025; date of current version 27 October 2025. This work was supported in part by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity Research and Development Program under Grant NCRP25-P04-TAICeN; in part by the Defence Science Organisation (DSO) Laboratories under the AI Singapore Program (AISG) under Award AISG2-GC-2023-008; in part by the National Research Foundation, Prime Minister's Office, Singapore, through the Campus for Research Excellence and Technological Enterprise (CREATE) Program; and in part by the National Natural Science Foundation of China under Grant 62502075. The associate editor coordinating the review of this article and approving it for publication was Dr. Abdallah Shami. (*Corresponding author: Hangcheng Cao.*)

Cong Wu is with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, also with the Exponential Science, Grand Cayman KY1-1106, Cayman Islands, and also with the Centre for Blockchain Technologies, University College London, WC1E 6BT London, U.K. (e-mail: cnacwu@whu.edu.cn).

Hangcheng Cao is with the Department of Computer Science, City University of Hong Kong, Hong Kong, China (e-mail: hangccao@cityu.edu.hk).

Jing Chen and Xiyu Yan are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: chenjing@whu.edu.cn; xiyuyan@whu.edu.cn).

Guowen Xu is with the School of Computer Science and Engineering, of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: guowen.xu@uestc.edu.cn).

Ziming Zhao is with the Khoury College of Computer Sciences, Northeastern University, Boston, MA 02115 USA (e-mail: z.zhao@northeastern.edu).

Yang Liu is with the Cyber Security Laboratory, College of Computing and Data Science, Nanyang Technological University, Singapore 639798, and also with MetaTrust Labs, Singapore 297722 (e-mail: yangliu@ntu.edu.sg).

Hongbo Jiang is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410012, China (e-mail: hongbojiang@hnu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3620669

approaches lack the flexibility to capture subtle transactional anomalies indicative of rugpulls. These limitations highlight the need for adaptive, data-driven solutions capable of transaction analysis. Such methods are essential for strengthening DeFi security and effectively addressing the rapidly changing landscape of financial fraud.

B. Motivation

Graph Neural Networks (GNNs) and their variants are increasingly recognized as a promising approach for analyzing complex relational data, particularly for detecting rugpull tokens [16], [22], [25], [26], [27]. Their ability to process graph-structured data, such as blockchain transaction networks, allows for a nuanced understanding of transaction dynamics crucial in identifying potential frauds. However, applying GNNs in this context is challenging due to several factors. Firstly, DeFi transactions exhibit rapid temporal dynamics, requiring models that can adapt effectively to these changes [5]. Most existing GNN approaches focus on static GNNs or semi-temporal GNNs, which combine static GNNs with LSTM layers to handle temporal information [28]. While these methods have been beneficial, they often struggle with the fast-evolving nature of DeFi transactions. Secondly, the sophisticated and often subtle nature of rugpull schemes necessitates advanced feature extraction and learning techniques [15], [19]. Additionally, the constantly evolving strategies of malicious actors in DeFi require GNN models that are dynamic and adaptable, capable of learning and evolving in response to new tactics and patterns [29], [30], [31]. These challenges highlight the need for advanced, flexible GNN models specifically for the DeFi environment to effectively mitigate the risks of rugpull tokens.

C. RUGSCREENER

In this paper, we introduce RUGSCREENER, a novel framework specifically designed to forecast tokens at risk of being rugpulled within the DeFi ecosystem, before fraudulent activity is fully executed. Unlike traditional methods that often detect rugpulls after they have occurred, RUGSCREENER employs a dynamic, temporal graph-based approach to model blockchain transactions, capturing the evolving and complex interactions among tokens over time. This predictive capability is crucial for detecting rugpull risks, enabling stakeholders to take preventive actions before any significant damage occurs. RUGSCREENER leverages advanced memory mechanisms and attention-based learning to analyze transaction patterns and detect anomalies, focusing on key transactional relationships and providing a historical context. By enriching its analysis with node attributes such as transaction frequency and centrality measures, RUGSCREENER enhances its ability to identify potential rugpulls proactively. This approach not only capitalizes on the strengths of GNNs in managing graph-structured data but also addresses the specific challenges inherent in the DeFi environment, positioning RUGSCREENER as a sophisticated and versatile tool for safeguarding investments in the rapidly evolving DeFi landscape. Through its comprehensive analysis of transaction behaviors

and the intentions of initiators, RUGSCREENER offers a proactive solution for detecting rugpull tokens effectively.

D. Novelty of RUGSCREENER

Unlike conventional rugpull detection methods [14], [17], [19] and prior GNN-based approaches [25], [26], [32], RUGSCREENER introduces a fully temporal graph neural network tailored for rugpull detection in DeFi. Rather than relying on post-event signals or static models, RUGSCREENER performs proactive analysis by embedding time as a learnable component directly within the GNN architecture. This design enables the model to capture evolving transaction dynamics more effectively than hybrid approaches that combine static GNNs with RNNs or LSTMs. Furthermore, RUGSCREENER incorporates a rich set of node features, along with temporal attention and memory mechanisms, to enhance representation learning in rapidly changing environments. These innovations address the limitations of semi-temporal models and offer a robust, scalable framework for predicting fraudulent behavior in decentralized financial systems.

E. Technical Insights

RUGSCREENER tackles rugpull detection by modeling token transactions as a temporal graph, where each node represents an address enriched with structural and behavioral features such as centrality and transfer frequency. To capture evolving patterns, the model uses a temporal graph neural network enhanced with memory and attention mechanisms. The memory component retains historical node states, allowing the model to learn long-term behavioral trends, while the attention mechanism selectively aggregates relevant neighbor information to emphasize salient patterns. Unlike prior methods that treat time as a static input or use external recurrent layers, RUGSCREENER encodes time directly as a learnable vector, enabling fine-grained temporal reasoning. This design improves the model's adaptability to irregular transaction intervals and short-lived fraudulent behaviors. Together, these components enable RUGSCREENER to maintain a high-resolution view of each token's transactional context, improving the precision and timeliness of rugpull detection. By jointly modeling structural, temporal, and behavioral signals, RUGSCREENER overcomes the limitations of static and semi-temporal models and advances the state of fraud detection in decentralized finance.

In summary, we make the following contributions:

- 1) We propose RUGSCREENER, the first approach to apply a fully temporal graph neural network for rugpull token detection. It leverages attention-based message passing and memory-driven temporal modeling to track token transaction dynamics and assess rugpull risk.
- 2) To enable robust temporal graph learning, we enrich node attributes by analyzing node centrality and transfer frequency. We also design several temporal graph learning strategies utilizing memory mechanism-based historical representation aggregation and attention-based neighbor nodes' representation aggregation, thus learning robust representations from temporal transactions.

- 3) We compile and open-source the first standard dataset for rugpull token prediction, including both token transactions and labels. This dataset consists of an unlabeled dataset with 1,882,114 transactions from 29,595 tokens for graph learning and a labeled dataset with 128,819 transactions from 500 benign and 500 rugpull tokens for evaluation.
- 4) We comprehensively evaluate the performance of RUGSCREENER under various settings and present typical case studies and insights. It achieves a balanced accuracy (BAC) of 95.7% in detecting rugpull tokens, outperforming existing state-of-the-art temporal graph learning models.

F. Code Availability

We aim to promote open, reproducible, and transparent research among the academic research community. We publish the dataset and codebase of RUGSCREENER, which can be obtained at GitHub repository: <https://github.com/yxy036/RugScreener-main>.

II. BACKGROUND

This section introduces background of rugpull and GNN.

A. DeFi and Rugpull Tokens

DeFi is reshaping the financial domain by employing blockchain to enable intermediary-free transactions, leading to the emergence of decentralized lending, borrowing, and complex financial tools [5], [22]. DeFi's operations hinge on smart contracts, with ERC20 tokens being of particular prominence [12]. These tokens, conforming to a standard protocol on the Ethereum blockchain, offer varied capabilities from digital asset representation to utility tokens and governance rights within decentralized applications (DApps). Smart contracts facilitate token transfers with terms coded directly into them, offering a high level of transparency and security in contract management and enforcement.

DeFi leverages Automated Market Makers (AMMs) and liquidity pools to enable decentralized asset trading [33]. AMMs replace traditional order books with algorithmic pricing, while liquidity pools allow users to lock assets in smart contracts to facilitate swaps. Liquidity providers earn transaction fees proportional to their contribution. Despite this innovation, DeFi remains vulnerable to rugpulls, scams in which developers withdraw liquidity or dump tokens, causing sharp price collapses and eroding user trust [19].

Rugpulls typically begin with the launch of a new token and liquidity pool on a DeFi platform, often promoted through aggressive tactics such as wash trading and pump-and-dump schemes [6], [33]. These strategies artificially inflate the token's value to attract investors, after which the developers withdraw liquidity, causing the token to collapse in value. Common red flags include unrealistic return promises, anonymous or unverifiable teams, opaque tokenomics, and smart contracts with hidden withdrawal functions. Recent rugpulls increasingly exhibit short token lifespans, centralized token supply, and little to no community or technical activity.

Identifying these behavioral patterns is critical for designing effective detection and prevention mechanisms.

The analysis of rugpull risks in transaction data is fraught with challenges. The pseudonymity of DeFi transactions allows fraudsters to hide their tracks, while the high volume of activity can obscure the signs of manipulation [12]. Changes in DeFi protocols further complicate pattern recognition, and without centralized oversight, the burden of detecting rugpulls falls on decentralized community efforts. This environment demands advanced, adaptable tools capable of sifting through complex data to pinpoint potential fraud. Recent rugpull cases, such as \$CTA, \$CREDIX, \$Quill, and \$GOW39 on BSCchain, Ethereum, and Scroll, highlight the urgency. For instance, token \$Quill¹ on Scroll saw its deployer remove a large amount of liquidity on October 28, 2023, causing the currency price to drop by 92.43%. Similarly, token \$CTA² on BNB Chain experienced a price drop of 96.85% on November 28, 2023, further exemplifying the critical need for sophisticated detection mechanisms.

B. Graph Neural Network

GNNs have ushered in a new era of data analysis for complex relational structures, such as those found in social networks, molecular frameworks, and logistical networks, offering profound insights into their interconnected nature [27]. Unlike conventional neural networks that are tailored for data with a regular grid structure, such as images or text, GNNs along with their derivatives, including GraphSAGE [34], GAT [35], and TGAT [26], are adept at decoding the intricate web of connections in graph-based data. These connections are represented by nodes and the edges between them. The ability of GNNs to adeptly capture node representations in such networks plays a pivotal role in blockchain analytics, particularly for the identification of rugpull tokens where the patterns of transactions between nodes can be telling of potential fraudulent schemes.

The core idea of GNN and its variants lies in the principle of aggregating both the intrinsic attributes of nodes and the contextual information encoded in their interconnections. Using the adjacency matrix A and the feature matrix X , GNNs iteratively refine node representations through message-passing. The update rule for the node embeddings H can be expressed as

$$H^{(k)} = \sigma(AH^{(k-1)}W^{(k)} + B^{(k)})$$

where $H^{(k)}$ is the matrix of node embeddings at iteration k , $W^{(k)}$ are learnable weight matrices, $B^{(k)}$ are bias terms, and σ is an activation function. While GNNs are effective at capturing relational structures by aggregating information from neighboring nodes, they face key limitations in DeFi contexts. Most notably, standard GNNs lack mechanisms to model temporal dependencies, which are critical in DeFi where transaction patterns evolve rapidly. Without temporal modeling, GNNs struggle to adapt to shifting behaviors and detect emerging rugpull risks in time-sensitive data streams.

¹scroll:0 × 786214Ee3EB0fE1d96AbBbaaADb06E869036f862

²bsc:0xcc7a9b5e0b20f1363a79544c2ef84ce7bf50ab57

III. PROBLEM FORMULATION

Rugpulls are a prevalent scam in the DeFi ecosystem, where developers fraudulently abandon a project after raising significant funds, leaving investors with worthless tokens. These schemes can manifest in various forms, such as sudden liquidity withdrawals, high-frequency token transfers, and abnormal token minting patterns. In our work, we specifically target rugpulls manipulated by fraudsters, where they intentionally dump tokens or remove liquidity, often after running the scam for varying durations, from just one day to extended periods. The common thread in these cases is the fraudulent intent to deceive investors by artificially inflating the token's value and then abruptly exiting the market, causing the token value to plummet and leaving investors with substantial losses. This type of rugpull poses significant financial risks and undermines trust in the DeFi ecosystem, making its detection critical.

Given the dynamics of token transactions, they can be represented as a temporal graph $G = (V, E, T, X_v, X_e)$. In this representation, V denotes the set of nodes, with each node representing an address, and E represents the set of edges, indicating transactions between two addresses. The temporal dynamics are captured by T , which associates timestamps with each edge. Additionally, X_v and X_e are the feature vectors associated with nodes and edges, respectively. Each node $v_i \in V$ is associated with a feature vector $x_i \in X_v$, representing the address's attributes. An edge $e_{ij} \in E$ between nodes v_i and v_j indicates a transaction and carries temporal information from T , having a feature vector from X_e . Our primary objective is to learn a robust representation using temporal graph learning to predict potential rugpull tokens.

The rugpull prediction task is a binary classification problem. For each token i , we aim to predict its label y_i , where $y_i = 1$ if it is involved in a rugpull and $y_i = 0$ otherwise. Our objective is to minimize the loss function $\mathcal{L}(\theta)$ across all tokens, formulated as $\mathcal{L}(\theta) = \sum_{i=1} \mathcal{L}_i(y_i, \hat{y}_i)$, where $\mathcal{L}_i$ denotes the loss for the prediction of token i , y_i is the true label, $\hat{y}_i$ is the predicted label, and θ represents the model parameters. To address this problem, we design a temporal GNN model tailored for learning representations in a dynamic graph to predict future rugpull risks. This model, $\mathcal{F}_\theta$, parameterized by θ , generates an embedding for each token i as h_i , capturing both the structural and temporal characteristics of token transactions within the temporal graph, formulated as $h_i = \mathcal{F}_\theta(G(V, E, T, X_v, X_e))$. The final prediction $\hat{y}_i$ is derived by passing h_i through a binary detector to assess the risk of a rugpull event, $\mathcal{D}$, i.e., $\hat{y}_i = \mathcal{D}(h_i)$.

IV. SYSTEM DESIGN

This section illustrates system overview and each module.

A. System Overview

As shown in Figure 2, RUGSCREENER consists of four modules. It begins with the data collector, which gathers token transfer data from blockchain blocks to build a detailed transaction history. This data is processed by the temporal graph builder, which normalizes and enriches node attributes by analyzing centrality and transfer frequency, creating an

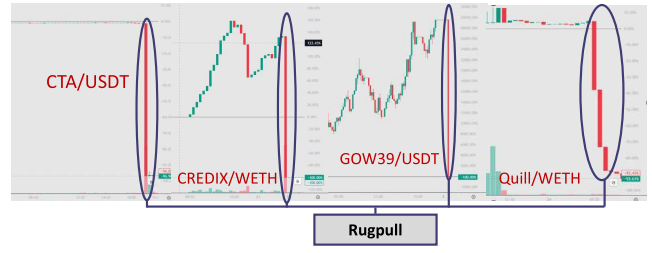


Fig. 1. Examples of rugpull tokens in real world.

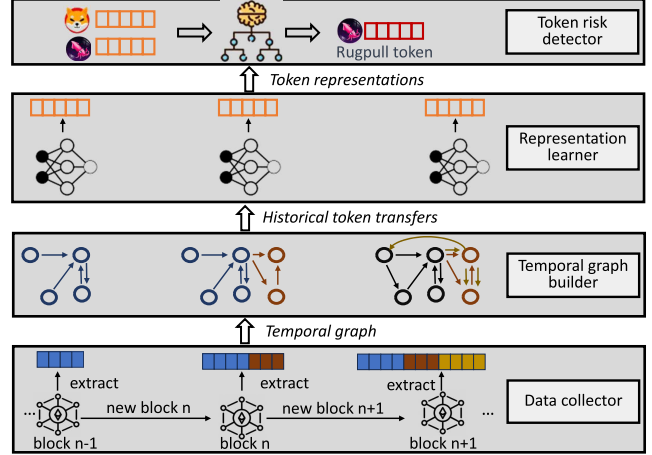


Fig. 2. Overview of RUGSCREENER.

evolving graph that captures the dynamic interactions among addresses over time. The representation learner then employs a temporal learning model, enhanced by memory and attention mechanisms, to distill this transactional data into meaningful token representations, highlighting potential fraud indicators. Finally, the token risk detector uses classifiers to analyze these representations and identify tokens with a high probability of future rugpull risk. This integrated framework provides predictive insights, enabling stakeholders to protect their digital assets against fraud in the decentralized financial ecosystem.

B. Data Collector

We collected Ethereum transaction data using Web3.py [36], a Python library that interacts with Ethereum nodes to access on-chain data in real time. Specifically, we extracted Transfer events emitted by ERC-20 token contracts [37], capturing essential fields including sender and receiver addresses, transfer values, transaction hashes, log indices, and timestamps. To ensure consistency and reliability, we configured the extraction process to validate data integrity and avoid event duplication. The resulting dataset captures granular, time-ordered token movement across addresses, serving as a foundational resource for modeling transactional behavior and identifying patterns indicative of anomalous or malicious activity.

C. Temporal Graph Builder

This module preprocesses transactions and extracts node attributes to construct the temporal token transfer graph.

1) *Preprocessing*: During preprocessing, we standardized transaction timestamps and converted alphanumeric addresses

into numerical indices to simplify data handling and reduce storage needs. We applied z-score normalization to transaction values for consistency across tokens. We also restructured transaction records into dual graph edges, representing transfers from sender to token and token to receiver, ensuring the data was ready for temporal graph analysis.

2) *Node Attributes Extraction*: Extracting node attributes is essential for building the temporal graph of Ethereum token transfers. Raw data lacks inherent attributes for nodes, so we assign them to add depth and context. This uncovers the roles and interactions of nodes, providing insight into their significance within the network. By extracting network-based features, we offer a detailed understanding of each node's behavior and relationships.

- 1) *In-degree* represents the number of edges directed to a node, it signifies the count of incoming transactions for a node. Formulated as: $ID(v) = |\{u \in V | (u, v) \in E\}|$, where V is the set of nodes and E is the set of edges.
- 2) *Out-degree* indicates the number of edges originating from a node, reflecting the count of outgoing transactions. Defined as: $OD(v) = |\{w \in V | (v, w) \in E\}|$.
- 3) *Degree centrality* measures the importance of a node based on the number of connections it has to other nodes. Calculated as: $DC(v) = \frac{ID(v) + OD(v)}{|V| - 1}$.
- 4) *Closeness centrality* indicates how close a node is to all other nodes in the network. Computed as: $CC(v) = \frac{|V| - 1}{\sum_{u \in V} d(u, v)}$, where $d(u, v)$ is the shortest path distance between nodes u and v .
- 5) *Betweenness centrality* reflects the extent to which a node lies on the shortest paths between other nodes. Formulated as: $BC(v) = \sum_{s, t \in V} \frac{\sigma(s, t | v)}{\sigma(s, t)}$, where $\sigma(s, t)$ is the total number of shortest paths from node s to node t and $\sigma(s, t | v)$ is the number of those paths passing through node v .
- 6) *Eigenvector centrality* determines a node's influence based on the centrality of its neighbors. It is calculated iteratively, based on the centrality of adjacent nodes.
- 7) *Incoming transfer frequency* for a node measures the rate at which it receives transactions. It is defined as $itf(i) = \frac{\text{NumIncomingTrans}(i)}{\text{TotalTime}}$, where $itf(i)$ is the incoming transfer frequency for node i , $\text{NumIncomingTrans}(i)$ is the number of incoming transactions to node i , and TotalTime is the total observation time period.
- 8) *Outgoing transfer frequency* for a node quantifies the rate at which it initiates transactions. It is defined as $otf(i) = \frac{\text{NumOutgoingTrans}(i)}{\text{TotalTime}}$, where $otf(i)$ is the outgoing transfer frequency for node i , $\text{NumOutgoingTrans}(i)$ is the number of outgoing transactions from node i , and TotalTime is the total observation time period.

The intuition behind these features lies in their ability to detect patterns indicative of rugpull scams in the Ethereum network. The in-degree and out-degree attributes, which count incoming and outgoing transactions, can reveal abnormal activities like sudden spikes in outgoing transfers, often associated with rugpulls. Degree centrality measures a node's connectivity and can identify key transaction hubs, where a sudden drop in activity might signal a rugpull. Closeness centrality shows how quickly information spreads from a node;

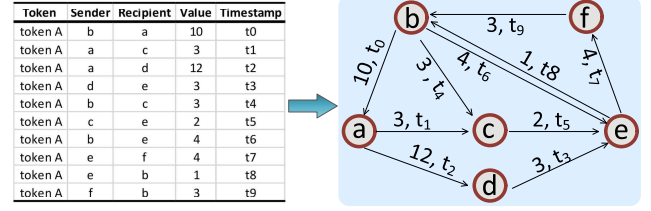


Fig. 3. Example of formulating transactions as temporal graph.

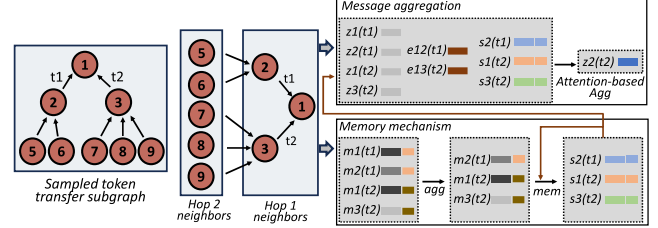


Fig. 4. Illustration of temporal graph learning.

a high initial centrality that quickly diminishes may indicate a scam unfolding. Betweenness centrality highlights nodes that often serve as intermediaries, with changes suggesting attempts to hide fraudulent actions. Finally, eigenvector centrality measures a node's influence based on its connections, where a sudden drop in activity could indicate a rugpull. Monitoring incoming and outgoing transfer frequencies can also help identify a rugpull, especially when these rates are unusually high.

By extracting and analyzing these attributes, RUGSCREENER effectively distinguishes between normal market fluctuations and fraudulent activities, enabling accurate prediction and mitigation of rugpull risks in the DeFi ecosystem. This detailed feature extraction provides a clear view of the network's structure, highlighting key nodes and transaction flows within the Ethereum ecosystem, thereby improving the prediction of rugpull scams. Figure 3 shows how token transfers are represented as a temporal graph, where nodes represent Ethereum addresses and directed edges capture transaction values and timestamps, illustrating the dynamic relationships involved.

D. Representation Learner

RUGSCREENER's representation learning process leverages the power of GNNs to analyze and predict token transactions by utilizing memory and attention mechanisms, which are critical for capturing the dynamic nature of token interactions over time [38], [39], [40]. The memory mechanism continuously updates the state of each node in the graph based on its transaction history, while the attention mechanism aggregates information from temporal neighbors to learn effective node representations. The model is trained in a self-supervised manner, predicting future transactions by analyzing past interactions. Figure 4 illustrates the steps of representation learning within a token transfer graph, depicting a subgraph with nodes (accounts) at times $t1$ and $t2$, organized into hops based on transactional distance. Each node's state $s_i(t)$ is updated using the memory mechanism, which aggregates messages $m_i(t)$ from prior interactions, while attention mechanisms

weigh the importance of temporal neighbors, thereby capturing and modeling the dynamic patterns of transactions.

1) *Memory-Based Historical Node's Representation Aggregation*: At each time step t , every node i maintains a memory state $s_i(t)$, which encodes its interaction history up to that point. This memory is updated whenever a transaction occurs. For a directed transaction event $e_{ij}(t)$ between nodes i and j , we compute two directional messages:

$$\mathbf{m}_i(t) = \text{msg}_s(s_i(t^-), s_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad (1)$$

$$\mathbf{m}_j(t) = \text{msg}_d(s_j(t^-), s_i(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad (2)$$

where $s_i(t^-)$ denotes the memory state of node i just before time t , and $\Delta t = t - t^-$ is the time elapsed since the node's last update. The functions msg_s , msg_d , and msg_n are parameterized message functions that learn how to encode temporal and contextual dependencies from node states and event features. For node-wise events $\mathbf{v}_i(t)$, a single message is computed:

$$\mathbf{m}_i(t) = \text{msg}_n(s_i(t^-), t, \mathbf{v}_i(t)). \quad (3)$$

Messages for the same node within a batch are aggregated to form a single message $\bar{m}_i(t)$ using operations like taking the most recent or mean of the messages:

$$\bar{m}_i(t) = \text{Agg}(m_i(t_1), \dots, m_i(t_n)). \quad (4)$$

Gated recurrent unit (GRU) model is employed to update the memory, integrating new information from recent interactions:

$$s_i(t) = \text{GRU}(\bar{m}_i(t), s_i(t^-)), \quad (5)$$

where $\bar{m}_i(t)$ is the aggregated message, $s_i(t^-)$ is the current memory state, and $s_i(t)$ is the updated state, reflecting the latest temporal dynamics.

Attention-based neighbor nodes' representation aggregation. We used the attention mechanism to aggregate neighbor nodes' representations [41]. To embed the temporal information in graph learning, the temporal representation is first learned by generating a temporal embedding $z_i(t)$ for node i at time t , using time projection and temporal graph attention. The graph attention layers compute node i 's embedding by aggregating information from its temporal neighbors:

$$z(t) = [\tilde{h}_0^{(l-1)}(t) \|\Phi_{d_r}(0), \tilde{h}_1^{(l-1)}(t_1) \|\Phi_{d_r}(t - t_1), \dots, \tilde{h}_N^{(l-1)}(t_N) \|\Phi_{d_r}(t - t_N)]^T, \quad (6)$$

where $\Phi(\bullet)$ is a temporal encoding function to project the time interval into a temporal embedding. We used the temporal encoding function in TGAT [26].

For the attention-based aggregating neighbor nodes, the query $q(t)$, key $K(t)$, and value $V(t)$ can be given as:

$$q(t) = [z(t)]_0 W_Q = (\tilde{h}_0^{(l-1)}(t) \|\Phi_{d_r}(0)) * W'_Q \quad (7)$$

where $[z(t)]_0$ and $\tilde{h}_0^{(l-1)}(t)$ is the generated embedding of current node at time t by previous graph layer.

$$\begin{aligned} K(t) &= [z(t)]_{1:N} W_K \\ &= (\tilde{h}_1^{(l-1)}(t_1) \|\Phi_{d_r}(t - t_1), \dots, \tilde{h}_N^{(l-1)}(t_N) \|\Phi_{d_r}(t - t_N)) * W'_K \end{aligned} \quad (8)$$

where $[z(t)]_{1:N}$ is the generated neighbor nodes' embedding by considering temporal embedding. $\tilde{h}_1^{(l-1)}(t_1)$ is the generated neighbor nodes' embedding at time t_1 by previous graph layer.

$$\begin{aligned} V(t) &= [z(t)]_{1:N} W_V \\ &= (\tilde{h}_1^{(l-1)}(t_1) \|\Phi_{d_r}(t - t_1), \dots, \tilde{h}_N^{(l-1)}(t_N) \|\Phi_{d_r}(t - t_N)) * W'_V, \end{aligned} \quad (9)$$

where W_Q , W_K , W_V are learnable parameters weight matrices. The attention weight can be formulated as:

$$\alpha_i = \frac{\exp(q^T K_i)}{\sum_q \exp(q^T K_q)}. \quad (10)$$

Then, the output of each attention head $h^{(i)}$ can be represented as:

$$h^{(i)} = \text{Attn}^{(i)}(q^{(i)}(t), K^{(i)}(t), V^{(i)}(t)), \quad (11)$$

Finally, we used a two-layer fully-connected network to learn the node embedding from different attention heads' outputs, which can be formulated as:

$$\tilde{h}_0^{(l)}(t) = \text{MLP}(h^{(1)}(t) \|\dots\| h^{(k)}(t)) \| x_0, \quad (12)$$

2) *Model Training*: The training phase of the graph model in RUGSCREENER is defined by a self-supervised learning approach, focusing on the prediction of future transactions based on historical transaction data. The effectiveness of this training is quantified using the Binary Cross-Entropy (BCE) loss function, which is specifically designed for binary classification tasks like predicting the existence of an edge (or transaction) in the network. The BCE loss function is mathematically expressed as:

$$\min - \frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (13)$$

where N is the number of samples, y_i is the true label (1 if the edge exists, 0 otherwise), and $\hat{y}_i$ is the predicted probability of the edge existing. The loss function aggregates the error across all samples, penalizing deviations of the predicted probabilities $\hat{y}_i$ from the actual labels y_i . The term inside the summation applies a logarithmic transformation to the predicted probabilities, which is a standard approach in logistic regression models.

Algorithm 1 outlines the training process for the graph model. Each node, representing an Ethereum address, maintains an evolving memory state that updates with each transaction. This update is managed by a memory mechanism that aggregates messages from previous interactions, while an attention mechanism assesses the relevance of neighboring nodes based on their temporal closeness. Training progresses over time, with each node's memory state and temporal embeddings being iteratively refined through learned message functions and GRUs. The model is trained in a self-supervised manner, aiming to predict future transactions based on past interactions, optimized using a binary cross-entropy loss. In RUGSCREENER, rugpull transactions are modeled using a temporal graph where vertices represent Ethereum addresses (e.g., token creators, liquidity providers, investors) and edges

Algorithm 1 Training Process of Graph Model**Input:** Graph $G(V, E, X, T)$ **Output:** Trained Model $\mathcal{M}$

```

1: Initialize  $\mathbf{s}_i(0) \forall i \in V$ 
2: Initialize  $\theta_{\text{msg}}, W_Q, W_K, W_V$ 
3: for  $t \in T$  do
4:   for  $i \in V$  do
5:      $\mathbf{m}_i(t) \leftarrow \text{msg}(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t))$ 
6:      $\bar{\mathbf{m}}_i(t) \leftarrow \text{Agg}(\{\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_n)\})$ 
7:      $\mathbf{s}_i(t) \leftarrow \text{GRU}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-))$ 
8:   for  $i \in V$  do
9:      $\mathbf{z}_i(t) \leftarrow \text{Attn}(\mathbf{s}_i(t), \mathcal{N}(i))$ 
10:   Predict  $\hat{y}$  based on  $\{\mathbf{z}_i(t)\}$ 
11:    $\mathcal{L} \leftarrow \text{BCE}(y, \hat{y})$ 
12:   Update  $\mathcal{M}$  using  $\nabla \mathcal{L}$ 
13: return  $\mathcal{M}$ 

```

signify transactions between them. Each vertex is enriched with attributes like in-degree, out-degree, centrality measures, and transaction frequency, capturing the role and influence of each address. Edges are defined by transaction-specific attributes such as value, timestamp, and direction, reflecting the flow of tokens over time.

E. Token Risk Detector

In the token risk prediction phase, we leverage the rich embeddings generated by our temporal GNN model, which encapsulate both the structural and temporal dynamics of token transactions within the blockchain. For each token, including those never seen before (i.e., unlabeled tokens), these embeddings serve as a comprehensive feature set that captures its behavior and interaction patterns. These embeddings are then fed into a classification pipeline where established machine learning techniques, such as support vector machines (SVM) and decision trees, are employed to evaluate and predict the likelihood of a token being involved in a rugpull. The transition from embedding to classification is seamless; the GNN-derived features are directly used as input for the classifiers, ensuring that even unlabeled tokens are analyzed based on the same rigorous criteria.

V. PERFORMANCE EVALUATION

This section reports performance results.

A. Experimental Setup

1) *Dataset*: We used two Ethereum ERC-20 token transaction datasets (2019-2022), each serving a distinct role in the evaluation of RUGSCREENER. The *unlabeled dataset*, containing 1,882,114 transactions from 29,595 tokens, is used for unsupervised temporal graph representation learning. In contrast, the *labeled dataset* comprises 128,819 transactions from 1,000 carefully selected tokens, 500 rugpull tokens and 500 benign tokens, and is used exclusively for downstream classification evaluation. These labeled tokens were chosen for their higher activity levels to ensure meaningful signal for the fraud detection task. Rugpull labels were obtained from

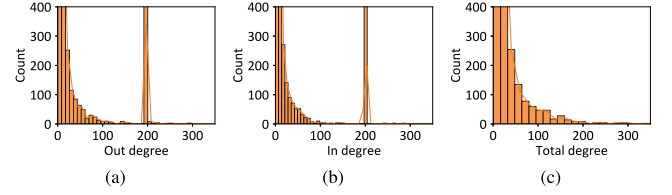


Fig. 5. Distribution of out degree (a), in degree (b), and total degree (c) of benign tokens.

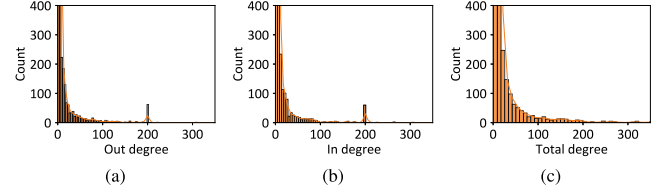


Fig. 6. Distribution of out degree (a), in degree (b), and total degree (c) of rugpull tokens.

TABLE I
DATA STATISTICS

	Training Set	Val. and Testing Set
# Nodes	188,460	72,949
# Edges	720,000	257,638
# Accounts	183,527	72,004
# benign Tokens	No Label	500
# Malicious Tokens	No Label	500
# Tokens	5,260	1,000
# benign Token Transactions	No Label	99,947
# Malicious Token Transactions	No Label	28,872
# Transactions	360,000	128,819

blockchain audit platforms Certik [42] and Chainagis [43], which maintain public threat intelligence on confirmed scam tokens based on on-chain analysis and post-incident reports. The different dataset sizes reflect their separate objectives and do not indicate any inconsistency.

We define a rugpull as a token in which the deployer or affiliated party intentionally removes liquidity or dumps tokens, causing a rapid, irreversible price collapse. To validate the accuracy of the labels, we manually inspected transaction patterns, focusing on behaviors such as abrupt liquidity withdrawal, developer abandonment, and price crashes inconsistent with broader market trends. Benign tokens were selected to match rugpull tokens in transaction volume, supply, and market capitalization, ensuring a balanced and fair comparison. The dataset was partitioned into training, validation, and testing sets in a 70%-15%-15% split. Table I summarizes the key statistics, including the number of nodes, edges, accounts, and transactions used in each phase.

In addition to transaction records, we analyzed the network characteristics of tokens in the labeled dataset. Specifically, we examined the in-degree, out-degree, and total degree of both benign and rugpull tokens. For benign tokens, the in-degree, out-degree, and total degree were (3.19 ± 21.72) , (3.19 ± 23.65) , and (6.38 ± 44.82) , respectively, illustrated as Figure 5. For rugpull tokens, the in-degree, out-degree, and total degree were (5.30 ± 23.41) , (5.30 ± 23.59) , and (10.61 ± 46.39) , respectively, illustrated as Figure 6. These statistics highlight the differences in transaction patterns between

benign and malicious tokens, providing additional insights into their behavior. The labeled dataset contains 500 benign tokens and 500 rugpull tokens, with 99,947 benign token transactions and 28,872 malicious token transactions in the test set, detailed as Table I.

2) *Model Parameters and Optimization*: We optimized temporal graph learning and token risk prediction using the Adam optimizer with a learning rate of 0.0001 [44] and a batch size of 128. The model employed a dropout rate of 0.1 and used GRU for memory updates [45]. Key parameters included memory and time embedding dimensions, both set to 64. We experimented with varying the number of graph layers (1 or 2) and node embedding dimensions, selecting from {16, 32, 64, 100} for a single layer and {32, 50, 64} for two layers. The number of attention heads was tested between {3, 8}, and neighbor aggregation was evaluated with {5, 10, 15, 20} neighbors. We used equal negative sampling and neighbor sampling [25] to enhance model efficiency while maintaining accuracy. Cross-validation was employed to fine-tune parameters, with comparisons against the TGAT benchmark to assess the impact of node attributes.

3) *Metrics*: We evaluated the model using BAC, average precision (AP), F1 score, and area under the curve (AUC). BAC is the average of true positive rate and true negative rate, particularly useful for imbalanced datasets. AP measures the precision-recall curve, while the F1 score balances precision and recall. AUC indicates the model's ability to distinguish between positive and negative samples. Experiments were conducted on a system with an Intel Xeon CPU E5-2680 v4 @ 2.40GHz and an NVIDIA Tesla P4 GPU, running Ubuntu 18.04.5 LTS. The development environment included torch 1.10.0+cu111, dgl-cu111-0.9.1, networkx 3.1, scikit-learn 0.19.1, and imbalanced-learn 0.8.1, with CUDA 11.1 for GPU acceleration.

B. Impact of Embedding Dimension

The embedding dimension in GNNs is a critical parameter that determines the size of the feature vectors representing nodes in the graph. It directly influences the model's ability to capture and represent the complexities of the data. In our evaluation, we varied the embedding dimension to understand its impact on the model's performance. We experimented with dimensions {16, 32, 64, 100} when the number of graph layers was set to 1, and {32, 50, 64} for a configuration with 2 graph layers. This variation allowed us to observe the trade-offs between model complexity and performance.

1) *Evaluation with Single Graph Layer*: The evaluation results for a single graph layer, as detailed in Figure 7, reveal insightful trends about the impact of embedding dimensions on model performance. The optimal performance was observed with an embedding dimension of 64, achieving an AP of 95.4%, an AUC of 0.947, recall of 88.3%, FPR of 11.3%, and FNR of 11.7% on the test set. This represents a notable enhancement over smaller dimensions, such as 16 and 32, which recorded AP of 92.7% and 94.3%, and AUC values of 0.920 and 0.934, respectively. The improvement from dimension 16 to 32 is modest yet significant, indicating that a moderate increase in dimension can effectively enrich the

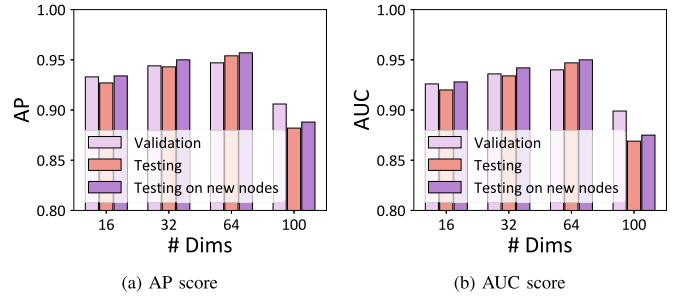


Fig. 7. AP and AUC under different dimensions of embeddings when neighborhood depth is 1.

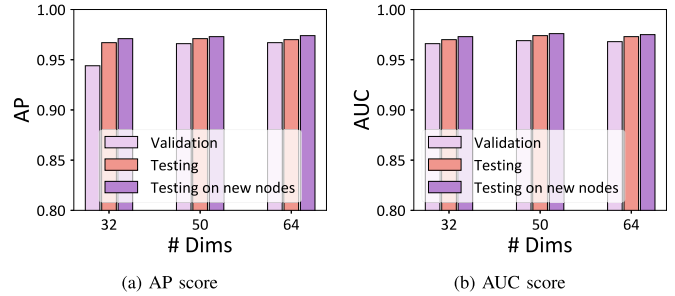


Fig. 8. AP and AUC under different dimensions of embeddings when neighborhood depth is 2.

model's representational capacity. However, a further increase to a dimension of 100 led to a marked decline, with AP and AUC dropping to 88.2% and 0.869 respectively. In fact, it could be indicative of overfitting, where the model becomes too tailored to the training data, losing its generalization ability.

2) *Evaluation with Two Graph Layers*: The addition of a second graph layer, as shown in Figure 8, significantly impacts the model's performance across different embedding dimensions. While an embedding dimension of 64 continues to perform well (AP: 97.0%, AUC: 0.973, recall: 93.6%, FPR: 2.2%, FNR: 6.4%), dimensions of 32 and 50 also deliver highly competitive results. Notably, the 50-dimensional setting achieves the highest AP and AUC scores (97.1% and 0.974, respectively), with recall at 91.2%, FPR at 7.9%, and FNR at 8.8%. This suggests that the additional graph layer enhances the model's ability to effectively utilize lower-dimensional embeddings, likely due to its improved capacity to capture complex feature interactions. Furthermore, the performance at a dimension of 32 shows a notable improvement compared to the single-layer setup (AP: 96.7% vs. 94.3%, AUC: 0.970 vs. 0.934), highlighting how the second layer provides richer feature representations and more effective learning even at smaller dimensions. These findings emphasize the importance of layer depth in graph GNNs, especially when balancing embedding dimensionality with computational efficiency.

C. Impact of Neighborhood Depth

Neighborhood depth in GNNs refers to the number of graph layers used in the model, determining the extent of information aggregation from a node's neighbors. We varied the neighborhood depth by changing the number of graph layers from one to two, allowing us to assess its impact

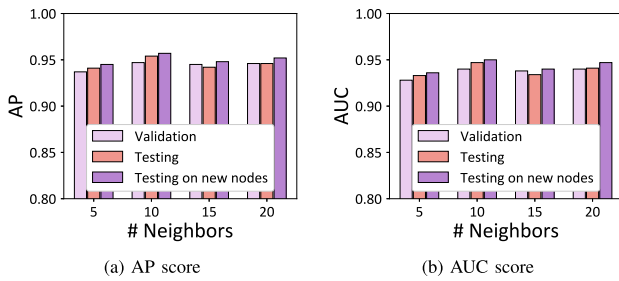


Fig. 9. AP and AUC under different neighborhood widths.

on model performance. Figure 7 shows that with a single graph layer, the model's peak performance was achieved at an embedding dimension of 64 (AP: 95.4%, AUC: 0.947). In contrast, Figure 8 illustrates that increasing the number of graph layers to two resulted in notable performance improvements, with the highest AP (97.1%) and AUC (0.974) achieved at a dimension of 50. This indicates that deeper information aggregation through increased neighborhood depth enhances the model's ability to capture complex node relationships, even with lower-dimensional embeddings. The consistent performance gains with additional layers underscore the importance of neighborhood depth in GNN design, particularly for tasks like token transaction analysis, where accurately modeling complex relational data is crucial.

D. Impact of Neighborhood Width

Neighborhood width, defined as the number of neighboring nodes sampled in GNNs, plays a crucial role in determining the model's performance. In our evaluation, we varied the neighborhood width to understand its impact on the model's ability to accurately predict token risks. Specifically, we experimented with different numbers of neighbors, including 5, 10, 15, and 20, to observe how the model's performance changes with varying degrees of neighborhood information. Figure 9 shows our model's performance across different neighborhood widths, measured by AP and AUC. With 5 neighbors, the model achieved an AP of 0.941 and an AUC of 0.933. Increasing the neighbors to 10 improved these metrics to an AP of 0.954 and an AUC of 0.947. However, further increases to 15 and 20 neighbors resulted in diminishing returns, with APs of 0.942 and 0.946 and AUCs of 0.934 and 0.941, respectively. These results indicate that while expanding the neighborhood initially boosts performance, there is a point beyond which additional neighbors offer little benefit, highlighting the need for a balanced selection to optimize efficiency and accuracy.

E. Impact of Attention Heads

Attention heads in GNNs play a crucial role in learning and representing complex relationships in data by independently attending to different parts of a node's neighborhood, capturing various aspects of the data. To evaluate their impact on predicting rugpull risks in token transactions, we experimented with 3 and 8 attention heads to assess their effect on model accuracy and robustness. As shown in Figure 10, increasing the number of attention heads led to a clear improvement in performance. With 3 attention heads, the model achieved an

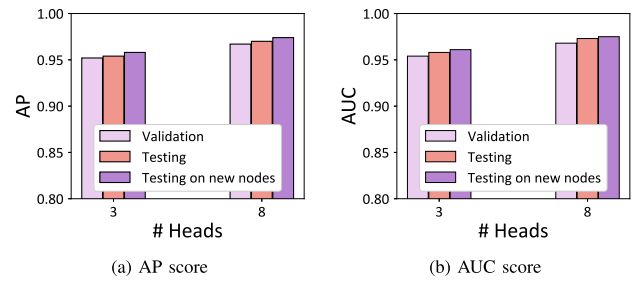


Fig. 10. AP and AUC under different attention heads.

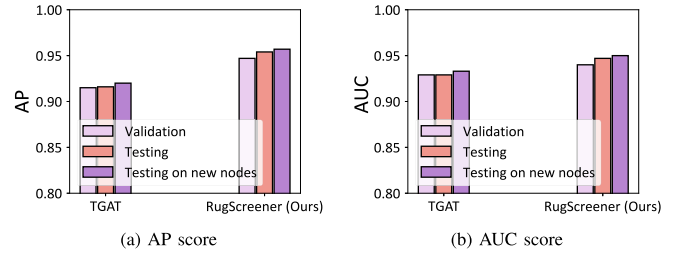


Fig. 11. Comparison of TGAT and RUGSCREENER (Ours).

AP of 0.954 and an AUC of 0.958 on the test set, while increasing to 8 heads significantly improved performance to an AP of 0.970 and an AUC of 0.973. This trend persisted on new nodes, where 8 heads achieved an AP of 0.974 and an AUC of 0.975, compared to 3 heads with an AP of 0.958 and an AUC of 0.961. These results underscore the importance of attention heads in capturing complex patterns in token transaction data, enabling more accurate and reliable predictions, and highlight their critical role in designing effective GNN architectures for tasks requiring an in-depth understanding of intricate data relationships, such as rugpull risk prediction.

F. Comparison With SOTA Graph Learning Model

TGAT [26] is a leading graph model known for its effectiveness in handling graph-structured data, especially when temporal dynamics are important. It uses an attention mechanism to track changes in graph data over time, making it an ideal benchmark for comparing with our RUGSCREENER. To assess its ability to predict rugpull risks in token transactions, we conducted a comparative analysis with TGAT, focusing on how our model advances and improves upon existing solutions.

Figure 11 presents our evaluation results, comparing the performance of RUGSCREENER with TGAT using AP and AUC metrics. On the test set, TGAT achieved an AP of 0.916 and an AUC of 0.929, while RUGSCREENER surpassed these with an AP of 0.954 and an AUC of 0.947. This superior performance was consistent across various evaluation sets, including validation and testing on new nodes. It achieved AP scores of 0.947 and 0.957, and AUC scores of 0.940 and 0.950, respectively, compared to TGAT's AP scores of 0.915 and 0.920, and AUC scores of 0.929 and 0.933. These results underscore the model's effectiveness in predicting rugpull risks, showcasing its ability to better capture temporal dynamics and complex relationships within the data, thereby advancing GNN-based financial security and fraud detection in DeFi platforms.

TABLE II
BAC AND F1 SCORE UNDER DIFFERENT SAMPLING METHODS, CLASSIFIERS, AND MODELS WITH OR WITHOUT NODE FEATURES

	Sampling	Method	AdaBoost				DTC				GBDT				KNN				SVC			
			Test		New nodes		Test		New nodes		Test		New nodes		Test		New nodes		Test		New nodes	
			BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1	BAC	F1
w/ node attributes	Raw data	TGAT	0.939	0.930	0.947	0.941	0.950	0.940	0.952	0.947	0.970	0.965	0.978	0.976	0.906	0.895	0.899	0.888	0.910	0.901	0.895	0.883
		Ours	0.953	0.946	0.973	0.970	0.949	0.940	0.950	0.945	0.976	0.971	0.983	0.981	0.912	0.902	0.890	0.877	0.908	0.898	0.895	0.883
	Random oversampling	TGAT	0.947	0.946	0.951	0.950	0.954	0.955	0.955	0.955	0.971	0.971	0.981	0.981	0.914	0.908	0.891	0.880	0.916	0.909	0.887	0.873
		Ours	0.958	0.958	0.964	0.964	0.950	0.951	0.953	0.952	0.968	0.967	0.981	0.981	0.920	0.914	0.882	0.871	0.916	0.909	0.887	0.873
	Smote sampling	TGAT	0.937	0.935	0.955	0.955	0.948	0.949	0.953	0.953	0.954	0.953	0.977	0.977	0.901	0.892	0.906	0.898	0.903	0.893	0.904	0.894
		Ours	0.952	0.952	0.962	0.962	0.945	0.944	0.957	0.956	0.964	0.963	0.981	0.981	0.906	0.898	0.897	0.889	0.903	0.893	0.904	0.894
	Random undersampling	TGAT	0.944	0.943	0.951	0.950	0.957	0.957	0.951	0.951	0.960	0.960	0.974	0.974	0.904	0.897	0.897	0.888	0.907	0.898	0.895	0.883
		Ours	0.955	0.954	0.967	0.967	0.965	0.965	0.956	0.955	0.971	0.970	0.981	0.981	0.912	0.905	0.893	0.883	0.907	0.898	0.895	0.883
	Comprehensive sampling	TGAT	0.940	0.938	0.955	0.954	0.942	0.940	0.961	0.960	0.950	0.948	0.978	0.978	0.926	0.921	0.922	0.916	0.926	0.920	0.922	0.915
		Ours	0.951	0.950	0.972	0.972	0.957	0.957	0.978	0.978	0.961	0.960	0.980	0.980	0.919	0.913	0.930	0.926	0.915	0.908	0.930	0.925
w/o node attributes	Raw data	TGAT	0.715	0.672	0.700	0.687	0.708	0.663	0.676	0.638	0.738	0.702	0.711	0.678	0.705	0.685	0.678	0.685	0.742	0.700	0.720	0.681
		Ours	0.731	0.689	0.704	0.665	0.630	0.582	0.644	0.600	0.719	0.675	0.682	0.645	0.705	0.683	0.695	0.699	0.739	0.691	0.714	0.661
	Random oversampling	TGAT	0.718	0.708	0.711	0.693	0.681	0.676	0.626	0.591	0.740	0.742	0.742	0.734	0.700	0.729	0.669	0.709	0.752	0.742	0.716	0.694
		Ours	0.739	0.728	0.690	0.667	0.691	0.687	0.645	0.621	0.714	0.708	0.692	0.677	0.708	0.735	0.690	0.725	0.744	0.725	0.712	0.671
	Smote sampling	TGAT	0.727	0.717	0.714	0.694	0.649	0.636	0.624	0.593	0.740	0.736	0.701	0.685	0.714	0.740	0.684	0.723	0.760	0.748	0.718	0.691
		Ours	0.735	0.721	0.727	0.706	0.653	0.642	0.658	0.649	0.719	0.711	0.692	0.681	0.702	0.725	0.701	0.738	0.754	0.730	0.727	0.687
	Random undersampling	TGAT	0.713	0.693	0.703	0.668	0.691	0.676	0.689	0.656	0.731	0.728	0.731	0.717	0.718	0.702	0.708	0.693	0.753	0.738	0.724	0.697
		Ours	0.739	0.723	0.710	0.687	0.652	0.649	0.675	0.652	0.697	0.689	0.709	0.691	0.718	0.699	0.727	0.705	0.726	0.703	0.724	0.681
	Comprehensive sampling	TGAT	0.748	0.739	0.737	0.719	0.693	0.682	0.687	0.665	0.768	0.764	0.751	0.732	0.742	0.773	0.695	0.727	0.773	0.760	0.731	0.707
		Ours	0.750	0.738	0.731	0.715	0.682	0.665	0.673	0.658	0.750	0.743	0.735	0.722	0.734	0.754	0.701	0.737	0.779	0.755	0.745	0.711

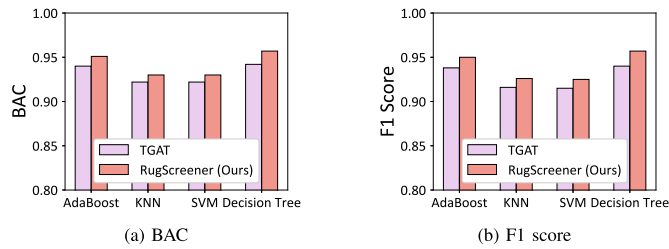


Fig. 12. BAC and F1 score under graph learning with node features.

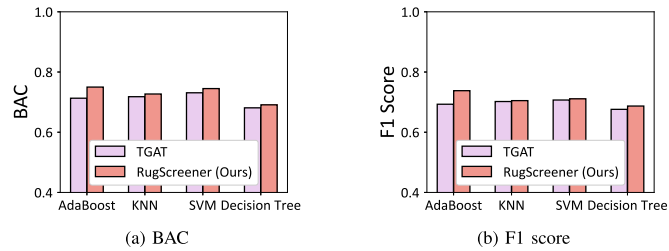


Fig. 13. BAC and F1 score under graph learning without node features.

G. Effectiveness of Node Attributes in Graph Learning

Node attributes in graph learning are critical features representing inherent characteristics of each node within a graph. These attributes significantly influence the learning process and the overall performance of graph-based models. In detecting rugpull tokens in Ethereum transactions, node attributes include various token-specific details such as transaction frequencies, token age, and other relevant metrics indicating suspicious activities. To evaluate the impact of these node attributes on the performance of our graph learning model, RUGSCREENER, we conducted experiments with and without incorporating node attributes and compared the results against the Temporal Graph Attention Network (TGAT) model. Figures 12 and 13 present the results of this evaluation, focusing on accuracy and F1 score as key performance metrics, across four different classifiers: AdaBoost, K-Nearest Neighbors (KNN), SVM, and Decision Tree.

The results in Figure 12 demonstrate that RUGSCREENER consistently outperforms TGAT across all classifiers when

TABLE III
COMPARISON OF EXISTING METHODS

Method	FPR (%)	FNR (%)	BAC (%)	F1 (%)
Mazorra et al. [17]	13.5	39.0	73.7	84.9
Zhou et al. [16]	14.4	21.1	82.3	87.9
Srifa et al. [11]	11.1	21.9	83.5	89.5
RUGSCREENER (Ours)	2.2	6.4	95.7	95.7

node attributes are included. For instance, with AdaBoost, RUGSCREENER achieved an accuracy of 0.951 and an F1 score of 0.950, surpassing TGAT's accuracy of 0.940 and F1 score of 0.938. This trend continues across all classifiers, underscoring the effectiveness of incorporating node attributes to enhance predictive capabilities. Conversely, Figure 13 shows a drop in performance for both models when node attributes are excluded, though RUGSCREENER still outperforms TGAT, albeit with a smaller margin (e.g., RUGSCREENER's accuracy of 0.750 and F1 score of 0.738 compared to TGAT's accuracy of 0.713 and F1 score of 0.693 with AdaBoost). These findings emphasize the critical role of node attributes in graph learning, particularly for tasks like rugpull detection, and highlight RUGSCREENER's robustness in handling complex data, even without additional context from node attributes.

H. Comparison With Existing Rugpull Detection Methods

We also compare RUGSCREENER with the latest rugpull detection methods from Mazorra et al. [17], Zhou et al. [16], and Srifa et al. [11]. Table III presents the performance comparison under RUGSCREENER and existing works. RUGSCREENER significantly outperforms these prior works across all metrics. Specifically, RUGSCREENER achieves a low FPR of 2.2% and a very low FNR of 6.4%, demonstrating its ability to minimize both false positives and false negatives compared to the competitors. Furthermore, our method achieves the highest BAC of 95.7%, which are notably higher than those of Mazorra et al. (73.7% BAC and 84.9% F1), Zhou et al. (82.3% BAC and 87.9% F1), and Srifa et al. (83.5% BAC and 89.5% F1).

I. Effectiveness Under Different Sampling Methods

During the training of the token risk detector, we used various data sampling techniques to address class imbalance and improve performance. We evaluated methods such as random oversampling [46], SMOTE [47], random undersampling [48], and SMOTETomek-based comprehensive sampling [49]. As shown in Table II, our method achieved the highest BAC and F1 scores with comprehensive sampling and node attributes (BAC: 0.951, F1: 0.950 on test data; BAC: 0.972, F1: 0.972 on new nodes). This trend was consistent across all sampling methods, highlighting the critical role of effective data sampling in boosting model accuracy.

Different sampling techniques yielded varied results, with random oversampling and SMOTE notably improving performance. For instance, random oversampling led to a BAC of 0.958 and an F1 of 0.958 on the test data, demonstrating the model's adaptability to balanced datasets. Even in scenarios without node attributes, our method maintained high accuracy, achieving a BAC of 0.750 and an F1 of 0.738 in comprehensive sampling. Throughout these evaluations, our method consistently outperformed TGAT across all sampling techniques, underscoring its robustness and the importance of appropriate sampling strategies in enhancing machine learning model performance.

J. Case Study

GemholicECO token,³ a rugpull token on the zkSync layer-2 network, serves as a notable case study by RUGSCREENER. In April 2023, the project locked 921 ETH on zkSync, giving an impression of security. However, after a zkSync upgrade that allowed access to these funds, the project quickly withdrew the 921 ETH and moved it across multiple chains. Simultaneously, GemholicECO's social media accounts were deleted, raising alarms. Community members, particularly NSerec from Zkmarkets, identified the scam, estimating losses at \$3.5 million. Despite the project's Know Your Customer (KYC) verification with SolidProof, the verification service has not responded to the situation, potentially harming its credibility.

ARKY token⁴ on the Ethereum network, detected by RUGSCREENER and subsequently reported to MetaTrust, serves as another illustrative case study in rugpull detection. This fraudulent token, which shares its name with a legitimate one, experienced a dramatic price drop of 100%, effectively wiping out all investor value. Fraudsters exploited the popularity of the genuine ARKY token to deceive investors by creating a fake version to attract unsuspecting buyers. This case highlights the tactics used by scammers to capitalize on trending assets and underscores the necessity for advanced tools like RUGSCREENER to detect and prevent fraud.

K. Effectiveness Under Varying Transaction Thresholds

To evaluate RUGSCREENER's rugpull detection accuracy at different stages of a token's lifecycle, we monitored performance across the first 20, 50, 100, 200, 300, and 500

transactions. The model achieves 88.5% accuracy within the first 20 transactions, which is improved to 92.3% by using 50 transactions, and 95.7% by using 200 transactions. Accuracy continued to rise, reaching 97.1% by the 500th transaction. These results highlight RUGSCREENER's effectiveness in detecting rugpull risks, providing crucial time for stakeholders in the DeFi ecosystem to take preventive action.

L. Takeaways of RUGSCREENER and Rugpull Tokens

1) *Transaction Behaviors*: Our model targets key transaction behaviors linked to rugpull schemes, such as sudden large withdrawals where developers liquidate holdings and abandon the project. High transfer frequencies, with frequent token transfers between accounts in a short period, can indicate manipulation and rugpull preparation. Unusual token minting patterns, where developers excessively mint tokens, dilute the token's value and are common in rugpull schemes.

2) *Node Attributes*: Node attributes reveal the roles and behaviors of entities in the blockchain network. Key attributes include in-degree and out-degree, showing the number of incoming and outgoing transactions. High in-degree with low out-degree suggests token accumulation by potential victims, while high out-degree with low in-degree indicates suspicious distribution patterns by the perpetrator. Centrality measures, such as degree centrality, closeness centrality, and betweenness centrality, identify influential nodes. Nodes with high centrality values are crucial in transaction flows and can signal suspicious activities. Sudden spikes in transaction frequencies also flag nodes for further analysis.

3) *Temporal Dynamics*: Understanding transactions' temporal aspects is crucial for effective fraud detection. The model employs time series analysis to detect anomalies, e.g., sudden bursts of activity, by aggregating temporal features that capture evolving behaviors and interactions. This temporal approach is instrumental in identifying patterns indicative of rugpull schemes and allows to adapt to new patterns and evolving tactics of malicious actors. By combining behavioral indicators with node attributes, the model gains structural insights into key players involved in fraudulent activities while also detecting chronological trends that static analysis might overlook.

VI. RELATED WORK

The identification of scam tokens has emerged as a sophisticated field, encompassing transaction-based [14], [19], [50] and contract-based approaches [12], [19], [22], [51]. Transaction-based methods focus on analyzing token transactions, with the foundational assumption that token contracts have built-in data structures for mapping addresses to token holdings and enabling token transfers. This approach, spearheaded by researchers like Victor and Lüders [21], involves constructing token networks where nodes represent addresses holding ERC-20 tokens and edges are token transfers between addresses. This methodology is crucial for uncovering transaction patterns and anomalies indicative of fraud.

In parallel, the security and vulnerabilities of smart contracts have garnered significant attention. This research spectrum ranges from analyzing transaction graphs and clustering malicious addresses to scrutinizing smart contract vulnerabilities.

³0xc651cf5dd958b6d7e4c417f1f366659237c34166

⁴0xe7593f6439dac6b6c252de4ec4ea6287859d71bc

A notable study by Xia et al. [14] concentrated on scam token identification within the Ethereum blockchain, especially in the Uniswap context [50]. Their method, although effective in identifying scam tokens, primarily serves as a post-event classification tool, lacking preemptive detection capabilities for fraudulent events like rugpulls. Recent studies further expand the DeFi threat landscape: DeFort [3] presents an automated system for detecting price manipulation attacks; SecPLF [24] addresses oracle manipulation vulnerabilities in loan protocols; Piercing the Veil of TVL [2] critiques the reliability of TVL as a DeFi security metric; and Reap the Harvest on Blockchain [4] surveys security and manipulation risks in yield farming protocols. These studies highlight the multifaceted risks in DeFi and underscore the importance of proactive, transaction-level detection techniques such as those proposed in our work.

Furthering this research, Mazonra et al. [17] expanded the dataset and introduced a machine learning algorithm for scam token detection, offering a detailed taxonomy of rugpull strategies and proposing enhanced graph representations. Cernera et al. [19] extended the investigation to tokens on both Ethereum and BNB Smart Chain, reinforcing the need for scalable, cross-platform detection tools. Contract-based methods, such as those by Liang et al. [22], focus on analyzing smart contract bytecode to identify malicious patterns in token contracts. A hybrid approach that combines transaction-based and contract-based features has been explored by Frowis et al. [51], enabling more comprehensive detection strategies. Chen et al. [12] further enriched this direction by constructing token ecosystem-wide transaction graphs, facilitating high-level fraud pattern discovery.

Recent work by Srifa et al. [11] introduced a machine learning framework that leverages time-series features such as transaction volume and count to detect rugpulls on decentralized exchanges like Uniswap. Their findings suggest that many rugpulls can be identified within a short time after token creation, emphasizing the value of early-stage transaction monitoring. Complementing this, Alhaidari et al. [52] presented SolRPDS, the first large-scale public rugpull dataset for the Solana blockchain. Covering over 3.6 billion transactions and 62,000 liquidity pools, SolRPDS offers extensive annotations on inactivity, liquidity changes, and interaction patterns, key indicators of rugpull behavior. This dataset opens the door for rigorous, data-driven research on Solana, which has historically lacked labeled datasets and targeted detection methods. Together, these studies highlight the growing recognition of rugpull risks across multiple blockchains and motivate the development of adaptive, graph-based models like RUGSCREENER for timely fraud detection in DeFi.

VII. DISCUSSION

The opaque nature of GNNs makes explainability essential for trust in rugpull detection. To improve transparency, we focus on analyzing key features that influence predictions and use attention mechanisms to highlight critical parts of the transaction graph. Additionally, post-hoc analysis may help clarify how specific transactions affect the model's decisions. These methods enhance both the accuracy and interpretability, making it more reliable for use in the DeFi ecosystem.

We acknowledge that the dataset (2019-2022) may not capture the latest rugpull tactics, such as memecoins and emerging fraud patterns from 2023-2024. However, RUGSCREENER's core framework is designed to adapt to new data, allowing it to incorporate evolving trends in DeFi. By periodically retraining the model with fresh data and integrating real-time monitoring, the system can stay up-to-date with emerging scams. This flexibility ensures that RUGSCREENER remains effective in detecting evolving fraud tactics in a rapidly changing DeFi landscape.

The comparison in Table III is not entirely fair, as the methods by Mazonra et al., [17], Zhou et al., [16] and Srifa et al. [11] target different datasets, labeling methods, and target variables. These works primarily rely on static graphs, non-temporal features, and binary classification on aggregated behaviors, which limits their ability to capture the evolving nature of DeFi fraud. Additionally, they often depend on heuristics or manually engineered features, which are constrained by the specific characteristics of the datasets they use. In contrast, RUGSCREENER utilizes a temporal graph neural network with memory and attention mechanisms, enabling it to model dynamic transaction patterns and detect subtle behavioral shifts. Our method directly learns from transactional sequences and node interactions, minimizing reliance on domain-specific feature engineering and offering greater adaptability.

A key limitation is its specificity to the Ethereum blockchain, limiting its use on other platforms like Binance Smart Chain, Polkadot, or Solana. Adapting it to these blockchains would require significant modifications, including recalibrating parameters, changing data extraction methods, and retraining on new datasets. Developing a modular framework that abstracts core functionalities could allow for easier customization across different blockchains. Additionally, leveraging transfer learning could speed up the adaptation and training process, improving the flexibility and cross-platform applicability [53].

The designs in RUGSCREENER, e.g., memory and attention mechanisms, introduce significant computational overhead, potentially limiting scalability and real-time detection capabilities. This challenge is particularly evident in environments with limited processing power. To optimize computational efficiency, strategies like model compression (e.g., pruning, quantization, and knowledge distillation) can be employed to reduce model size and improve inference speed with minimal accuracy loss. Additionally, using hardware accelerators like GPUs or TPUs can enhance performance, allowing for more efficient processing of large transaction volumes. Future work might also explore distributed computing frameworks to further reduce the computational load, ensuring scalability in real-world applications [54], [55], [56], [57], [58], [59], [60].

Rugpulls erode trust in decentralized finance, causing substantial financial losses and harming the reputation of blockchain technologies. By promptly identifying potential rugpulls, RUGSCREENER helps prevent these losses and shields investors from fraudulent activities. Beyond detecting threats, it contributes to the overall stability and integrity of the DeFi ecosystem, fostering a safer environment for

all participants. Emphasizing advanced adversarial detection mechanisms is crucial for sustaining the health and growth of decentralized financial markets [61].

RUGSCREENER detects rugpull risks by modeling transactional behavior through a temporal graph neural network, focusing on patterns such as sudden liquidity withdrawals and abnormal transfer activity. It enables decentralized exchanges (DEXs) to identify and flag suspicious tokens for potential delisting or investor warnings. Regulators may also leverage RUGSCREENER to trace fraudulent tokens and support enforcement actions. While our dataset spans 2019–2022, recent memecoin-related rugpulls exhibit distinct characteristics, such as short token lifespans and viral promotion. Although not included in the current dataset, RUGSCREENER's architecture supports retraining on new data, allowing adaptation to evolving fraud strategies in decentralized finance.

Our method begins with collecting and preprocessing blockchain transaction data, where timestamps are standardized, and addresses are converted into numerical indices. This processed data forms a temporal graph, with nodes representing Ethereum addresses and edges representing transactions, enriched with attributes like in-degree, out-degree, and centrality measures. The temporal graph is then fed into a GNN, which generates node embeddings that capture the network's structural and temporal dynamics. These embeddings are passed through a classification pipeline, where machine learning algorithms assess the risk of each token by detecting patterns indicative of rugpull risks. The system operates continuously, updating the graph and re-evaluating risks as new transactions occur, allowing for timely detection and prevention of fraudulent activities in the DeFi ecosystem.

In our study, we did not perform a direct comparison with existing DeFi attack detection methods due to several key factors. First, most existing methods focus on detecting rugpulls after they have occurred, while RUGSCREENER aims to forecast rugpull risks before they happen. This difference in objectives makes direct comparisons difficult. Additionally, inconsistent labeling criteria in previous studies complicate the alignment of our dataset with theirs. Many existing methods are also not open-source, and our attempts to reproduce their results were unsuccessful. Given these challenges, direct comparisons may be misleading and unfair. Instead, we highlight the novel aspects of RUGSCREENER and its effectiveness in proactively predicting rugpull risks.

VIII. CONCLUSION

In this paper, we introduced RUGSCREENER, a novel approach using temporal graph learning to detect rugpull risks in Ethereum token transactions. It leverages an effective graph learning model and incorporates key node attributes to capture complex transaction patterns that may indicate fraud. Our results show that it outperforms state-of-the-art models, offering superior precision and recall in identifying high-risk tokens. This highlights the potential of advanced graph learning techniques to enhance security within the DeFi ecosystem. We believe RUGSCREENER marks a significant step forward in using GNNs for detecting rugpull scams.

REFERENCES

- [1] R. Liang et al., "Vulseye: Detect smart contract vulnerabilities via stateful directed graybox fuzzing," 2024, *arXiv:2408.10116*.
- [2] Y. Luo, Y. Feng, J. Xu, and P. Tasca, "Piercing the veil of TVL: DeFi reappraised," 2024, *arXiv:2404.11745*.
- [3] M. Xie et al., "DeFort: Automatic detection and analysis of price manipulation attacks in DeFi applications," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Sep. 2024, pp. 402–414.
- [4] J. Xu and Y. Feng, "Reap the harvest on blockchain: A survey of yield farming protocols," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 1, pp. 858–869, Mar. 2023.
- [5] S. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. Knottenbelt, "SoK: Decentralized finance (DeFi)," in *Proc. 4th ACM Conf. Adv. Financial Technol.*, Sep. 2022, pp. 30–46.
- [6] J. Xu and B. Livshits, "The anatomy of a cryptocurrency pump-and-dump scheme," in *Proc. 28th USENIX Secur. Symp. (USENIX Secur.)*, 2019, pp. 1–18.
- [7] S. Chaliasos et al., "Smart contract and DeFi security tools: Do they meet the needs of practitioners?," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Feb. 2024, pp. 1–13.
- [8] Q. Kong, J. Chen, Y. Wang, Z. Jiang, and Z. Zheng, "DeFiTainter: Detecting price manipulation vulnerabilities in DeFi protocols," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 1144–1156.
- [9] L. Zhou et al., "SoK: Decentralized finance (DeFi) attacks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2444–2461.
- [10] R. Solomon, R. Weber, and G. Almashaqbeh, "SmartFHE: Privacy-preserving smart contracts from fully homomorphic encryption," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy (EuroS&P)*, Jul. 2023, pp. 309–331.
- [11] S. Srifa, Y. Yanovich, R. Vasilyev, T. Rupasinghe, and V. Amelin, "Rug pull detection on decentralized exchange using transaction data," *Blockchain: Res. Appl.*, vol. 6, no. 3, 2025, Art. no. 100275.
- [12] W. Chen, T. Zhang, Z. Chen, Z. Zheng, and Y. Lu, "Traveling the token world: A graph analysis of Ethereum ERC20 token ecosystem," in *Proc. Web Conf.*, Apr. 2020, pp. 1411–1421.
- [13] *Rug-Pull Scams Raked in Over U.S.\$2.8 Billion in Crypto in 2021*, Report Finds. Accessed: Dec. 15, 2023. [Online]. Available: <https://www.coindesk.com/tech/2023/12/27/crypto-users-lost-2b-to-hacks-scams-and-exploits-in-2023-defi-saysmargin/>
- [14] P. Xia et al., "Trade or trick?: Detecting and characterizing scam tokens on uniswap decentralized exchange," *ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 3, pp. 1–26, 2021.
- [15] S. Agarwal, G. Atondo-Siu, M. Ordekian, A. Hutchings, E. Mariconti, and M. Vasek, "Short paper: DeFi deception—Uncovering the prevalence of rugpulls in cryptocurrency projects," in *Financial Cryptography and Data Security*. Brač, Croatia: Springer, 2023.
- [16] Y. Zhou, J. Sun, F. Ma, Y. Chen, Z. Yan, and Y. Jiang, "Stop pulling my rug: Exposing rug pull risks in crypto token to investors," in *Proc. 46th Int. Conf. Softw. Eng., Softw. Eng. Pract.*, Apr. 2024, pp. 228–239.
- [17] B. Mazorra, V. Adan, and V. Daza, "Do not rug on me: Leveraging machine learning techniques for automated scam detection," *Mathematics*, vol. 10, no. 6, p. 949, Mar. 2022.
- [18] B. Mazorra, V. Adan, and V. Daza, "Do not rug on me: Zero-dimensional scam detection," 2022, *arXiv:2201.07220*.
- [19] F. Cernera, M. La Morgia, A. Mei, and F. Sassi, "Token spammers, rug pulls, and sniper bots: An analysis of the ecosystem of tokens in Ethereum and in the binance smart chain (BNB)," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1–19.
- [20] Z. Lin, J. Chen, J. Wu, W. Zhang, Y. Wang, and Z. Zheng, "CRPWarner: Warning the risk of contract-related rug pull in DeFi smart contracts," *IEEE Trans. Softw. Eng.*, vol. 50, no. 6, pp. 1534–1547, Jun. 2024.
- [21] F. Victor and B. K. Lüders, "Measuring Ethereum-based ERC20 token networks," in *Financial Cryptography and Data Security*. Frigate Bay, Saint Kitts and Nevis: Springer, 2019.
- [22] R. Liang et al., "PonziGuard: Detecting Ponzi schemes on Ethereum with contract runtime behavior graph (CRBG)," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Feb. 2023, pp. 1–12.
- [23] W. Chen et al., "SADPonzi: Detecting and characterizing Ponzi schemes in Ethereum smart contracts," *ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 2, pp. 1–30, 2021.
- [24] S. Arora, Y. Li, Y. Feng, and J. Xu, "SecPLF: Secure protocols for loanable funds against Oracle manipulation attacks," in *Proc. 19th ACM Asia Conf. Comput. Commun. Secur.*, Jul. 2024, pp. 1394–1405.
- [25] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," 2017, *arXiv:1709.05584*.

- [26] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," 2020, *arXiv:2002.07962*.
- [27] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [28] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 1–8.
- [29] S. Park, W. Jeong, Y. Lee, B. Son, H. Jang, and J. Lee, "Unraveling the MEV enigma: ABI-free detection model using graph neural networks," *Future Gener. Comput. Syst.*, vol. 153, pp. 70–83, Apr. 2024.
- [30] G. Yu, Q. Wang, T. Altaf, X. Wang, X. Xu, and S. Chen, "Predicting NFT classification with GNN: A recommender system for web3 assets," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2023, pp. 1–5.
- [31] M. Lee, G. Yu, and H. Dai, "Decentralized inference with graph neural networks in wireless communication systems," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2582–2598, May 2023.
- [32] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
- [33] J. Xu, K. Paruch, S. Cousaert, and Y. Feng, "SoK: Decentralized exchanges (DEX) with automated market maker (AMM) protocols," *ACM Comput. Surveys*, vol. 55, no. 11, pp. 1–50, Nov. 2023.
- [34] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [35] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [36] *Web3.Py*. Accessed: Dec. 15, 2023. [Online]. Available: <https://github.com/ethereum/web3.py>
- [37] *ERC20*. Accessed: Dec. 15, 2023. [Online]. Available: <https://docs.openzeppelin.com/contracts/3.x/erc20>
- [38] T. Tayeh, S. Aburakhia, R. Myers, and A. Shami, "An attention-based ConvLSTM autoencoder with dynamic thresholding for unsupervised anomaly detection in multivariate time series," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 2, pp. 350–370, 2022.
- [39] N. B. Puhan and S. S. Behera, "Holistic feature reconstruction-based 3-D attention mechanism for cross-spectral periocular recognition," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 435–448, 2023.
- [40] S. Hajra, M. Alam, S. Saha, S. Picek, and D. Mukhopadhyay, "On the instability of softmax attention-based deep learning models in side-channel analysis," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 514–528, 2024.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [42] *CertiK*. Accessed: Dec. 15, 2023. [Online]. Available: <https://www.certik.com/zh-CN>
- [43] *ChainAegis*. Accessed: Dec. 15, 2023. [Online]. Available: <https://app.chainaegis.com/>
- [44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [45] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [46] R. Mohammed, J. Rawashdeh, and M. Abdullah, "Machine learning with oversampling and undersampling techniques: Overview study and experimental results," in *Proc. 11th Int. Conf. Inf. Commun. Syst. (ICICS)*, Apr. 2020, pp. 243–248.
- [47] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [48] T. Hasanin and T. Khoshgoftar, "The effects of random undersampling with simulated class imbalance for big data," in *Proc. IEEE Int. Conf. Inf. Reuse Integr. (IRI)*, Jul. 2018, pp. 70–79.
- [49] T. Sasada, Z. Liu, T. Baba, K. Hatano, and Y. Kimura, "A resampling method for imbalanced datasets considering noise and overlap," *Proc. Comput. Sci.*, vol. 176, pp. 420–429, Jan. 2020.
- [50] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais, "High-frequency trading on decentralized on-chain exchanges," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 428–445.
- [51] M. Fröwis, A. Fuchs, and R. Böhme, "Detecting token systems on Ethereum," in *Financial Cryptography and Data Security*. Frigate Bay, Saint Kitts and Nevis: Springer, 2019.
- [52] A. Alhaidari, B. Kalal, B. Palanisamy, and S. Sural, "SolRPDS: A dataset for analyzing rug pulls in solana decentralized finance," 2025, *arXiv:2504.07132*.
- [53] C. Wu et al., "Rethinking membership inference attacks against transfer learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6441–6454, 2024.
- [54] M. Hu, T. Wei, M. Zhang, F. Mallet, and M. Chen, "Sample-guided automated synthesis for CCSL specifications," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2019, pp. 1–6.
- [55] M. Hu, M. Zhang, F. Mallet, X. Fu, and M. Chen, "Accelerating reinforcement learning-based CCSL specification synthesis using curiosity-driven exploration," *IEEE Trans. Comput.*, vol. 72, no. 5, pp. 1431–1446, May 2023.
- [56] M. Hu, E. Cao, H. Huang, M. Zhang, X. Chen, and M. Chen, "AloTML: A unified modeling language for AIoT-based cyber-physical systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 11, pp. 3545–3558, Nov. 2023.
- [57] M. Hu et al., "FedCross: Towards accurate federated learning via multi-model cross-aggregation," in *Proc. IEEE 40th Int. Conf. Data Eng. (ICDE)*, May 2024, pp. 2137–2150.
- [58] M. Hu et al., "FedMut: Generalized federated learning via stochastic mutation," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 11, pp. 12528–12537.
- [59] M. Hu et al., "GitFL: Uncertainty-aware real-time asynchronous federated learning using version control," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2023, pp. 145–157.
- [60] M. Hu et al., "Is aggregation the only choice? Federated learning via layer-wise model recombination," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 1096–1107.
- [61] C. Wu et al., "WAFBooster: Automatic boosting of WAF security against mutated malicious payloads," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1118–1131, Mar. 2025.